Module 12: Security and Best Practices

Comprehensive Security Framework

Security Architecture Overview

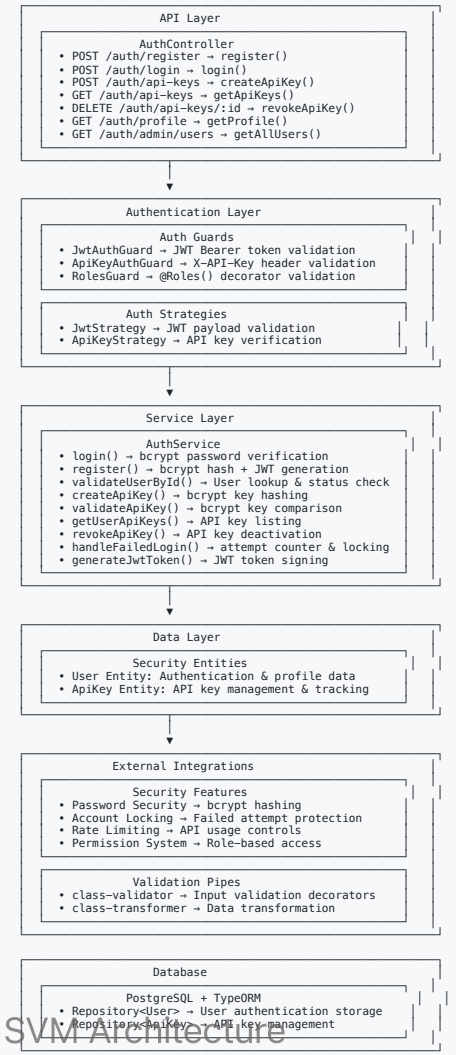
Multi-Layer Security

- **Authentication**: JWT tokens and API key validation
- **Authorization**: Role-based access control and permissions
- **Input Validation**: Comprehensive data sanitization
- **Rate Limiting**: API usage controls and abuse prevention
- **Audit Logging**: Complete security event tracking

Security Principles

- **Defense in Depth**: Multiple security layers
- **Least Privilege**: Minimum required permissions
- **Fail-Safe Defaults**: Secure default configurations
- **Complete Auditing**: All security events logged

System Architecture



Authentication Guards

JWT Authentication Guard

```
@Injectable()
export class JwtAuthGuard extends AuthGuard('jwt') {
  canActivate(context: ExecutionContext): boolean {
    // Allow access to auth endpoints without token
    const request = context.switchToHttp().getRequest();
    const isAuthEndpoint = request.url.includes('/auth/');

    if (isAuthEndpoint) {
      return true;
    }

    // Validate JWT token for protected endpoints
    return super.canActivate(context);
  }
}
```


API Key Authentication Guard

Authentication Strategies

JWT Strategy Implementation

```
@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor(private authService: AuthService) {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: process.env.JWT_SECRET,
    });
  }

  async validate(payload: JwtPayload): Promise<User> {
    const user = await this.authService.validateUserById(payload.sub);

    if (!user) {
      throw new UnauthorizedException('User not found');
    }

    if (user.status !== 'active') {
      throw new UnauthorizedException('User account is not active');
    }
    return user;
```

Security Entities

User Entity

```
@Entity()
export class User {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ unique: true })
  @Index()
  email: string;

  @Column()
  passwordHash: string;

  @Column({
    type: 'enum',
    enum: UserRole,
    default: UserRole.USER
  })
  role: UserRole;

  @Column({ default: 0 })
  loginAttempts: number;

  @Column({ nullable: true })
  lockedUntil?: Date;

  @Column({
    type: 'enum',
    enum: UserStatus,
    default: UserStatus.ACTIVE
  })
  status: UserStatus;

  @CreateDateColumn()
  createdAt: Date;

  @UpdateDateColumn()
  updatedAt: Date;

  // Business logic methods
  isLocked(): boolean {
    return this.lockedUntil && this.lockedUntil > new Date();
  }
```

Password Security

Password Hashing & Verification

```
@Injectable()
export class AuthService {
  private readonly SALT_ROUNDS = 12;

  async hashPassword(password: string): Promise<string> {
    return bcrypt.hash(password, this.SALT_ROUNDS);
  }

  async verifyPassword(password: string, hash: string): Promise<boolean> {
    return bcrypt.compare(password, hash);
  }

  async register(registerDto: RegisterDto): Promise<AuthResponse> {
    // Validate password strength
    this.validatePasswordStrength(registerDto.password);

    // Check if user exists
    const existingUser = await this.userRepository.findOne({
      where: { email: registerDto.email }
    });

    if (existingUser) {
      throw new ConflictException('User already exists');
    }

    // Hash password
    const passwordHash = await this.hashPassword(registerDto.password);

    // Create user
    const user = this.userRepository.create({
      email: registerDto.email,
      passwordHash,
      role: UserRole.USER
    });

    await this.userRepository.save(user);

    // Generate JWT token
    const token = this.generateJwtToken(user);
```

Account Protection

Failed Login Handling

```
async handleFailedLogin(email: string): Promise<void> {
  const user = await this.userRepository.findOne({ where: { email } });

  if (!user) {
    // Don't reveal if user exists
    return;
  }

  user.loginAttempts += 1;

  // Lock account after 5 failed attempts
  if (user.loginAttempts >= 5) {
    user.lockedUntil = new Date(Date.now() + 15 * 60 * 1000); // 15 minutes
    user.loginAttempts = 0; // Reset counter
  }

  await this.userRepository.save(user);
}

async login(loginDto: LoginDto): Promise<AuthResponse> {
  const user = await this.userRepository.findOne({
    where: { email: loginDto.email }
  });

  if (!user) {
    throw new UnauthorizedException('Invalid credentials');
  }

  // Check if account is locked
  if (user.isLocked()) {
    throw new UnauthorizedException('Account is temporarily locked');
  }

  // Verify password
  const isPasswordValid = await this.verifyPassword(loginDto.password, user.passwordHash);

  if (!isPasswordValid) {
    await this.handleFailedLogin(loginDto.email);
    throw new UnauthorizedException('Invalid credentials');
  }

  // Reset login attempts on successful login
  user.loginAttempts = 0;
  user.lockedUntil = null;
  await this.userRepository.save(user);

  // Generate token
```

API Key Management

API Key Creation & Validation

```
async createApiKey(userId: string, createDto: CreateApiKeyDto): Promise<ApiKeyResponse> {
  // Generate random API key
  const apiKeyValue = crypto.randomBytes(32).toString('hex');

  // Hash the full key for storage
  const keyHash = await bcrypt.hash(apiKeyValue, 12);

  // Store first 8 characters for identification
  const keyPrefix = apiKeyValue.substring(0, 8);

  // Create API key entity
  const apiKey = this.apiKeyRepository.create({
    keyHash,
    keyPrefix,
    user: { id: userId } as User,
    permission: createDto.permission || ApiKeyPermission.READ,
    expiresAt: createDto.expiresAt,
    rateLimit: createDto.rateLimit || 100
  });

  await this.apiKeyRepository.save(apiKey);

  // Return the actual key (only time it's shown)
  return {
    id: apiKey.id,
    key: apiKeyValue,  // Full key returned only once
    keyPrefix,
    permission: apiKey.permission,
    expiresAt: apiKey.expiresAt,
    rateLimit: apiKey.rateLimit
  };
}

async validateApiKey(apiKeyValue: string): Promise<ApiKeyValidationResult> {
  // Find API key by prefix (first 8 chars)
  const keyPrefix = apiKeyValue.substring(0, 8);

  const apiKey = await this.apiKeyRepository.findOne({
    where: { keyPrefix },
    relations: ['user']
  });

  if (!apiKey || !apiKey.isValid()) {
    return { isValid: false };
  }

  // Verify full key against hash
  const isKeyValid = await bcrypt.compare(apiKeyValue, apiKey.keyHash);

  if (!isKeyValid) {
    return { isValid: false };
  }

  return {
```

Rate Limiting & Usage Tracking

API Key Rate Limiting

```
async updateApiKeyUsage(apiKeyId: string): Promise<void> {
  const apiKey = await this.apiKeyRepository.findOne({
    where: { id: apiKeyId }
  });

  if (!apiKey) return;

  // Update usage statistics
  apiKey.usageCount += 1;
  apiKey.lastUsedAt = new Date();

  await this.apiKeyRepository.save(apiKey);
}

async checkRateLimit(apiKey: ApiKey): Promise<boolean> {
  if (!apiKey.lastUsedAt) {
    return true; // No usage yet
  }

  const now = new Date();
  const timeSinceLastUse = now.getTime() - apiKey.lastUsedAt.getTime();
  const requestsPerMs = apiKey.rateLimit / 60000; // per minute to per ms

  // Simple rate limiting: ensure minimum time between requests
  const minIntervalMs = 1000 / requestsPerMs;
```

Input Validation & Sanitization

DTO Validation with Class Validator

```
import { IsEmail, IsNotEmpty, MinLength, IsEnum, IsOptional } from 'class-validator';

export class RegisterDto {
  @IsNotEmpty()
  @IsEmail()
  email: string;

  @IsNotEmpty()
  @MinLength(8)
  password: string;

  @IsOptional()
  @IsEnum(UserRole)
  role?: UserRole;
}

export class LoginDto {
  @IsNotEmpty()
  @IsEmail()
  email: string;

  @IsNotEmpty()
  password: string;
}

export class CreateApiKeyDto {
  @IsOptional()
  @IsEnum(ApiKeyPermission)
  permission?: ApiKeyPermission;

  @IsOptional()
  expiresAt?: Date;
  @IsOptional()
  @Min(1)
```

API Endpoints

Authentication Endpoints

- `POST /auth/register` - User registration with password validation
- `POST /auth/login` - User login with account locking protection
- `GET /auth/profile` - Get current user profile
- `GET /auth/admin/users` - Admin endpoint for user management

API Key Management

- `POST /auth/api-keys` - Create new API key
- `GET /auth/api-keys` - List user's API keys
- `DELETE /auth/api-keys/:id` - Revoke API key

Security Configuration

Key Takeaways

Security Framework Benefits

- **Multi-Factor Authentication**: JWT + API key support
- **Comprehensive Protection**: Password security, account locking, rate limiting
- **Role-Based Access**: Hierarchical permission system
- **Complete Auditing**: All security events tracked

Implementation Best Practices

- **bcrypt Hashing**: Industry-standard password security
- **JWT Tokens**: Stateless authentication with expiration
- **Input Validation**: Prevent injection and malformed data
- **Rate Limiting**: Prevent abuse and DoS attacks
- **Audit Logging**: Complete security event tracking